[Lecture Note] Functions

MGS3001 Python Programming for Business, Spring 2025

Chad (Chungil Chae)

Mon, 26 May 2025

Learning Objectives

- Introduction to Functions
- Defining and Calling a Void Function
- Designing a Program to Use Functions
- Local Variables
- Passing Arguments to Functions
- Global Variables and Global Constants
- Introduction to Value-Returning Functions: Generating Random Numbers
- Writing Your Own Value-Returning Functions
- The math Module
- Storing Functions in Modules
- Turtle Graphics: Modularizing Code with Functions

SHORT VIDEO INTRODUCTION

https://www.youtube.com/watch?v=89cGQjB5R4M

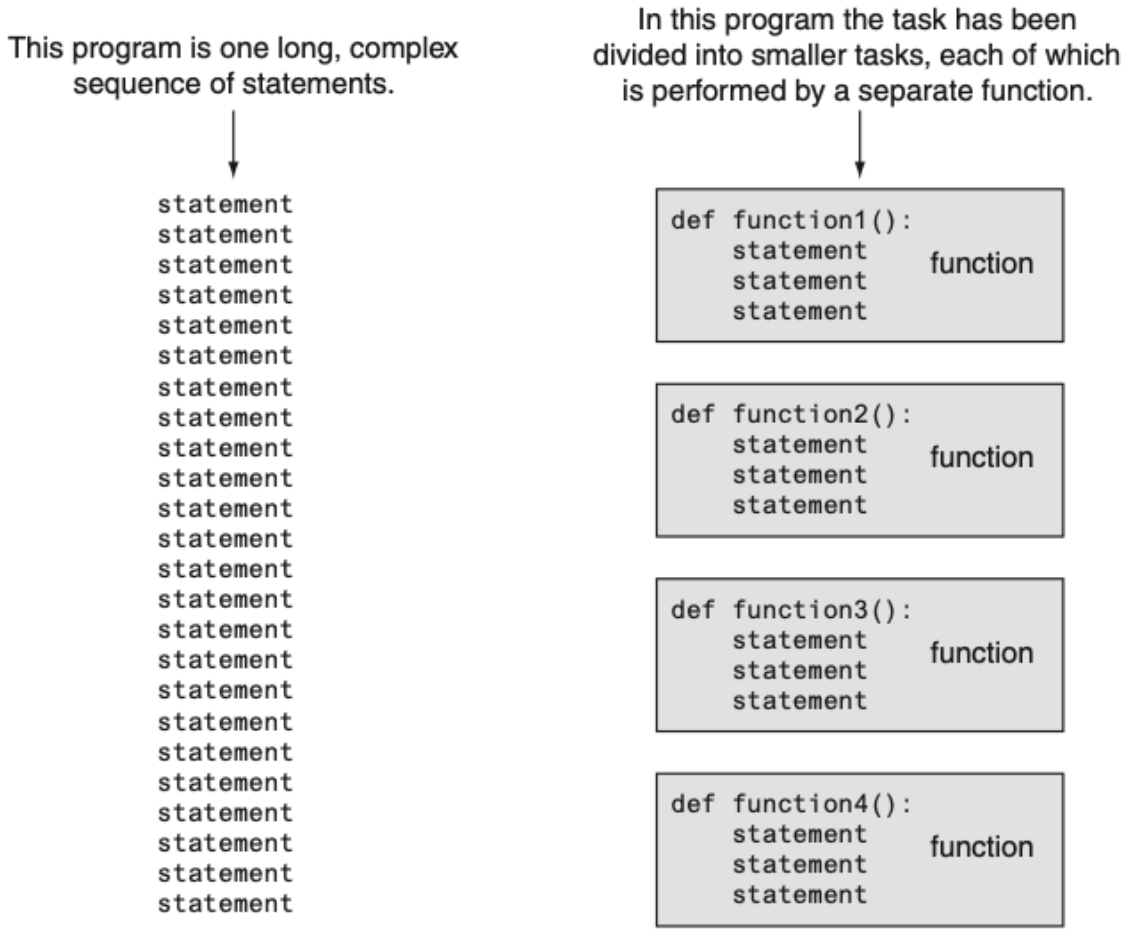
Introduction to Functions

A function is a group of statements that exist within a program for the purpose of performing a specific task.

- A more realistic payroll algorithm, the overall task of calculating an employee's pay would consist of several subtasks, such as the following:
  - Getting the employee's hourly pay rate

  - Getting the number of hours worked
  - Calculating the employee's gross pay
  - Calculating overtime pay
  - Calculating withholdings for taxes and benefits
  - Calculating the net pay
  - Printing the paycheck

- Most programs perform tasks that are large enough to be broken down into several subtasks.
- For this reason, programmers usually break down their programs into small manageable pieces known as functions.
- A function is a group of statements that exist within a program for the purpose of performing a specific task. Instead of writing a large program as one long



Benefits of Modularizing a Program with Functions

> **Simpler Code**
>
> A program's code tends to be simpler and easier to understand when it is broken down into functions. Several small functions are much easier to read than one long sequence of statements.

> **Code Reuse**
>
> Functions also reduce the duplication of code within a program. If a specific operation is performed in several places in a program, a function can be written once to perform that operation, then be executed any time it is needed. This benefit of using functions is known as code reuse because you are writing the code to perform a task once, then reusing it each time you need to perform the task.

> **Better Testing**
>
> When each task within a program is contained in its own function, testing and debugging becomes simpler. Programmers can test each function in a program individually, to determine whether it correctly performs its operation. This makes it easier to isolate and fix errors.

> **Faster Development**
>
> Suppose a programmer or a team of programmers is developing multiple programs. They discover that each of the programs perform several common tasks, such as asking for a username and a password, displaying the current time, and so on. It doesn't make sense to write the code for these tasks multiple times. Instead, functions can be written for the commonly needed tasks, and those functions can be incorporated into each program that needs them.

> **Easier Facilitation of Teamwork**
>
> Functions also make it easier for programmers to work in teams. When a program is developed as a set of functions that each performs an individual task,

Void Funcitons and Value-Returning Functions

- Two types of functions: **void functions** and **value-returning functions**.
  - Void function: it simply executes the statements it contains and then terminates.
  - Value-returning function: it executes the statements that it contains, then returns a value back to the statement that called it.
- Example
  - The input function -> a value-returning function.

    - When you call the input function, it gets the data that the user types on the keyboard and returns that data as a string.
  - The int and float functions -> value-returning functions.
    - You pass an argument to the int function, and it returns that argument's value converted to an integer.
    - Likewise, you pass an argument to the float function, and it returns that argument's value converted to a floating-point number.
    - The first type of function that you will learn to write is the void function.

Defining and Calling a Void Function

The code for a function is known as a function definition. To execute the function, you write a statement that calls it.

Function Names

- A function's name should be descriptive enough so anyone reading your code can reasonably guess what the function does.
- Python requires that you follow the same rules that you follow when naming variables, which we recap here:
  - You cannot use one of Python's keywords as a function name. (See Table 1-2 for a list of the keywords.)
  - A function name cannot contain spaces. The first character must be one of the letters a through z, A through Z, or an underscore character (_).
  - After the first character you may use the letters a through z or A through Z, the digits 0 through 9, or underscores.
  - Uppercase and lowercase characters are distinct.
- Because functions perform actions, most programmers prefer to use verbs in function names.
  - For example, a function that calculates gross pay might be named calculate_gross_pay.

Defining and Calling a Function

- To create a function, you write its definition. Here is the general format of a function definition in Python:


```
def function_name():
  statement
  statement
  etc
```

```
def message():
  print('I am Chad')
  print('Teaching Business Analytics Courses')
```

Call a function

- A function definition specifies what a function does, but it does not cause the function to execute.
- To execute a function, you must call it. This is how we would call the message function:

```
message()
```

- When a function is called, the interpreter jumps to that function and executes the statements in its block.
- Then, when the end of the block is reached, the interpreter jumps back to the part of the program that called the function, and the program resumes execution at that point.
- When this happens, we say that the function returns.

```
# Define funciton
def message():
  print('I am Chad')
  print('Teaching Business Analytics Courses')

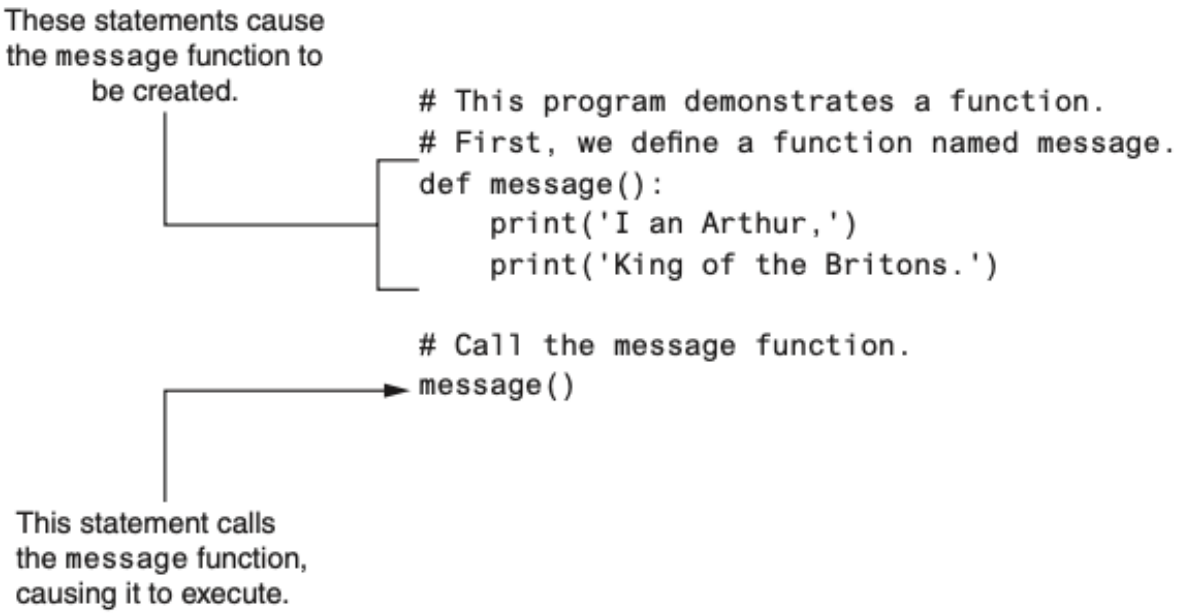
# Call the message function
message()
```

```
I am Chad
Teaching Business Analytics Courses
```

- First, the interpreter ignores the comments
- Then, it reads the def statement in line 2. This causes a function named message to be created in memory, containing the block of statements in lines 3 and 4.

- Remember, a function definition creates a function, but it does not cause the function to execute.

- Then it executes the statement in line 7, which is a function call.
  - This causes the message function to execute, which prints the two lines of output.



- Program 5-1 has only one function, but it is possible to define many functions in a program.
- In fact, it is common for a program to have a main function that is called when the program starts.
- The main function then calls other functions in the program as they are needed.
- It is often said that the main function contains a program's mainline logic, which is the overall logic of the program.

```
# Define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# next we define the message function.
def message():
    print('I am Chad,')
    print('A professor of MGS3001')

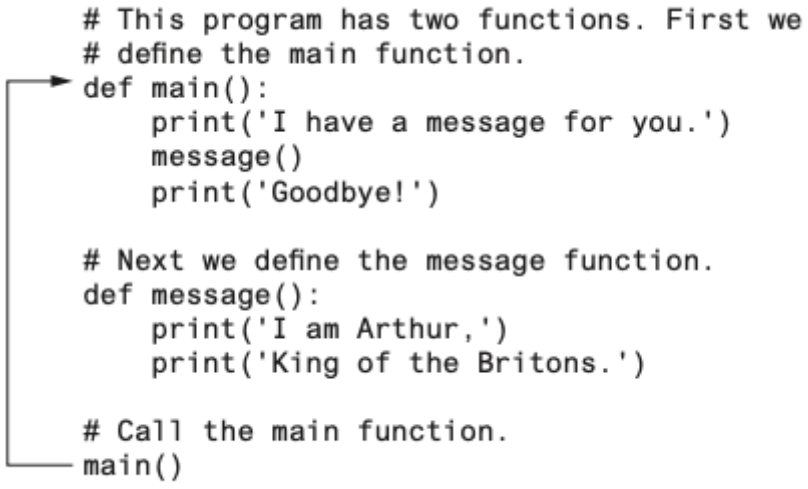
# Call the main function
main()
```



```
I have a message for you.
I am Chad,
A professor of MGS3001
Goodbye!
```

- The definition of the main function appears in lines 2 through 5, and the definition of the message function appears in lines 8 through 10. The statement in line 13 calls the main function

The interpreter jumps to the `main` function and begins executing the statements in its block.



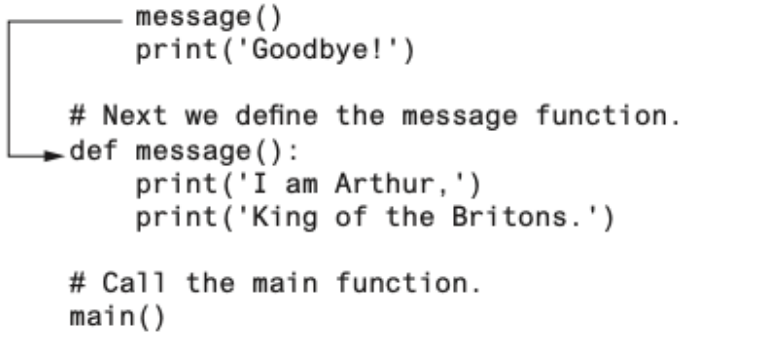
```
# This program has two functions. First we
# define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# Next we define the message function.
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# Call the main function.
main()
```

- The first statement in the main function calls the print function in line 4.
- It displays the string 'I have a message for you'.
- Then, the statement in line 5 calls the message function. This causes the interpreter to jump to the message function.
- After the statements in the message function have executed, the interpreter returns to the main function and resumes with the statement that immediately follows the function call.
- This is the statement that displays the string 'Goodbye!'.

The interpreter jumps to the `message` function and begins executing the statements in its block.



```
# This program has two functions. First we
# define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# Next we define the message function.
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# Call the main function.
main()
```


When the message function ends, the interpreter jumps back to the part of the program that called it and resumes execution from that point.

```
# This program has two functions. First we
# define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# Next we define the message function.
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# Call the main function.
main()
```

122

When the main function ends, the interpreter jumps back to the part of the program that called it. There are no more statements, so the program ends.

```
# This program has two functions. First we
# define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# Next we define the message function.
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# Call the main function.
main()
```

123

124 Indentation in Python

125 • In Python, each line in a block must be indented.
126 • The last indented line after a function header is the last line in the function's block.

The last indented line is the last line in the block.

These statements are not in the block.

```
def greeting():
    print('Good morning!')
    print('Today we will learn about functions.')

print('I will call the greeting function.')
greeting()
```

127

- When you indent the lines in a block, make sure each line begins with the same number of spaces. Otherwise, an error will occur.
- In an editor, there are two ways to indent a line:
  - (1) by pressing the Tab key at the beginning of the line, or
  - (2) by using the spacebar to insert spaces at the beginning of the line.
  - You can use either tabs or spaces when indenting the lines in a block, but don't use both.
    - Doing so may confuse the Python interpreter and cause an error.
- When you type the colon at the end of a function header, all of the lines typed afterward will automatically be indented.
- After you have typed the last line of the block, you press the Backspace key to get out of the automatic indentation.

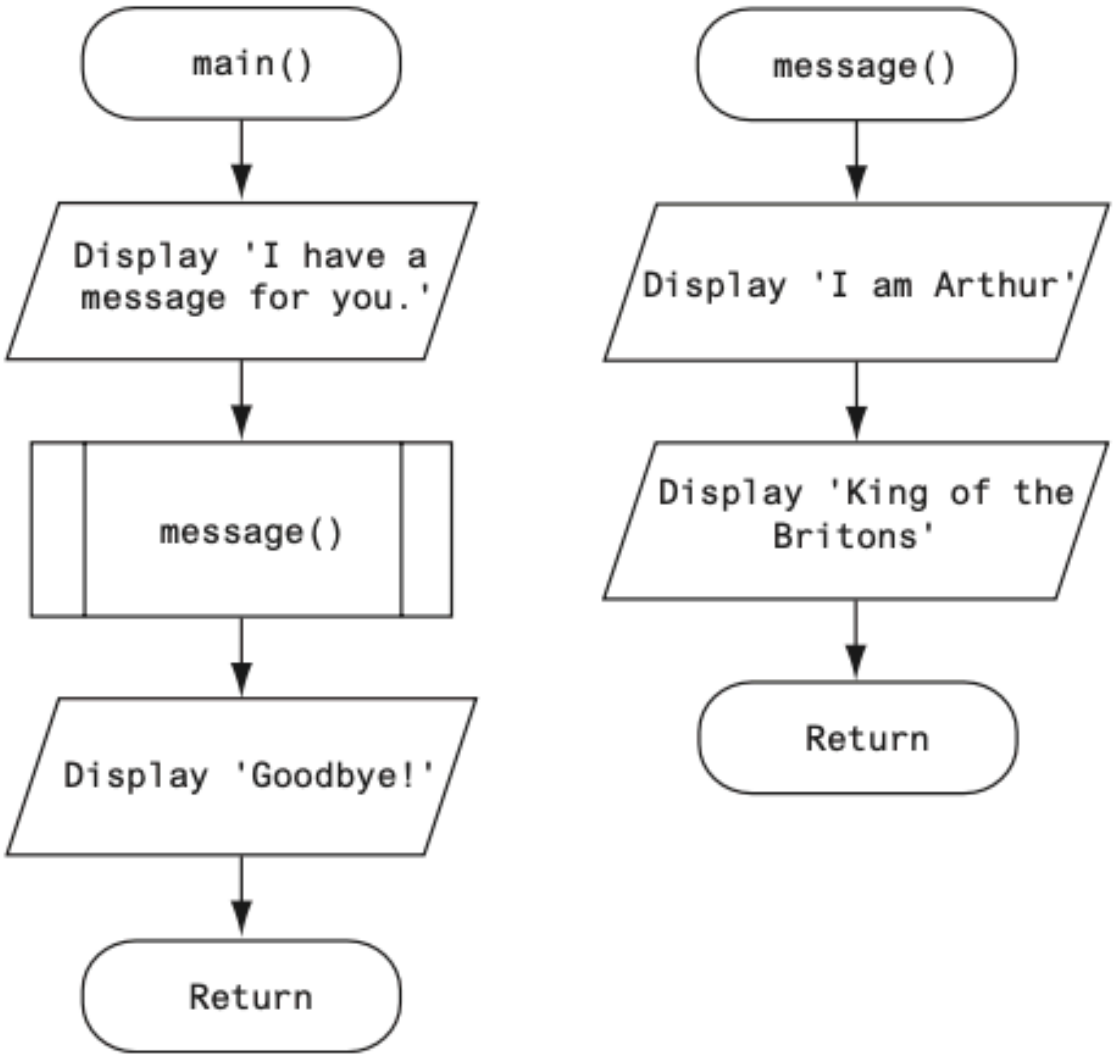
Designing a Program to Use Functions

Programmers commonly use a technique known as top-down design to break down an algorithm into functions.

Flowcharting a Program with Functions

- In a flowchart, a function call is shown with a rectangle that has vertical bars at each side
- The name of the function that is being called is written on the symbol.

- Programmers typically draw a separate flowchart for each function in a program.
  - When drawing a flowchart for a function, the starting terminal symbol usually shows the name of the function and the ending terminal symbol usually reads Return.



150

151 **Top-Down Design**

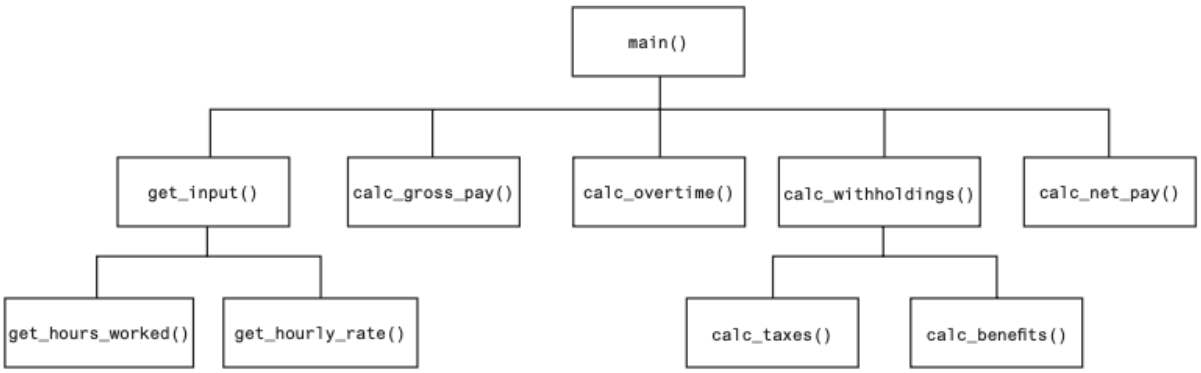
152 • Programmers commonly use a technique known as top-down design to break down an algorithm into
153 functions.

154 – The overall task that the program is to perform is broken down into a series of subtasks.
155 – Each of the subtasks is examined to determine whether it can be further broken down into more
156 subtasks. This step is repeated until no more subtasks can be identified.
157 – Once all of the subtasks have been identified, they are written in code.

158 This process is called top-down design because the programmer begins by looking at the top-
159 most level of tasks that must be performed and then breaks down those tasks into lower levels
160 of subtasks.

Hierarchy Charts

- Flowcharts are good tools for graphically depicting the flow of logic inside a function, but they do not give a visual representation of the relationships between functions.
- Programmers commonly use hierarchy charts for this purpose. A hierarchy chart, which is also known as a structure chart, shows boxes that represent each function in a program.
- The boxes are connected in a way that illustrates the functions called by each function.



Pausing Execution Unitil the User Presses Enter

- Sometimes you want a program to pause so the user can read information that has been displayed on the screen.
- When the user is ready for the program to continue execution, he or she presses the Enter key and the program resumes.
- In Python, you can use the input function to cause a program to pause until the user presses the Enter key.

Using the pass keyword

- Sometimes when you are initially writing a program's code, you know the names of the functions you plan to use, but you might not know all the details of the code that will be in those functions.
- When this is the case, you can use the pass keyword to create empty functions.
- Later, when the details of the code are known, you can come back to the empty functions and replace the pass keyword with meaningful code.

```
def step1():
  pass

def step2():
  pass
```



```
def step3():
  pass

def step4():
  pass
```

The pass keyword is ignored by the Python interpreter, so this code creates four functions that do nothing.

Local Variables

A local variable is created inside a function and cannot be accessed by statements that are outside the function. Different functions can have local variables with the same names because the functions cannot see each other's local variables.

- Anytime you assign a value to a variable inside a function, you create a local variable.
- A local variable belongs to the function in which it is created, and only statements inside that function can access the variable.
- The term local is meant to indicate that the variable can be used only locally, within the function in which it is created.
- An error will occur if a statement in one function tries to access a local variable that belongs to another function.

```
# This code generate error purposefully
# Definition of the main function.
def main():
    get_name()
    print('Hello', name) # This causes an error!

# Definition of the get_name function.
def get_name():
    name = input('Enter your name: ')

# Call the main function.
main()
```

- This program has two functions: main and get_name.
- The name variable is assigned a value that is entered by the user.
- This statement is inside the get_name function, so the name variable is local to that function.
- This means that the name variable cannot be accessed by statements outside the get_name function.
- The main function calls the get_name function.

- Then, the statement tries to access the name variable.
- This results in an error because the name variable is local to the get_name function, and statements in the main function cannot access it.

Scope and Local Variables

- A variable's scope is the part of a program in which the variable may be accessed.
- A variable is visible only to statements in the variable's scope.
- A local variable's scope is the function in which the variable is created.
- No statement outside the function may access the variable.
- In addition, a local variable cannot be accessed by code that appears inside the function at a point before the variable has been created.
- It will cause an error because the print function tries to access the val variable, but this statement appears before the val variable has been created.
- Moving the assignment statement to a line before the print statement will fix this error.

```
def bad_function():
  print(f'The value is {val}.') # This will cause an error!
  val = 99
```

- Because a function's local variables are hidden from other functions, the other functions may have their own local variables with the same name.

- In addition to the main function, this program has two other functions: texas and california. These two functions each have a local variable named birds.
- Although there are two separate variables named birds in this program, only one of them is visible at a time because they are in different functions.
- When the texas function is executing, the birds variable that is created in line 10 is visible.
- When the california function is executing, the birds variable that is created in line 15 is visible.

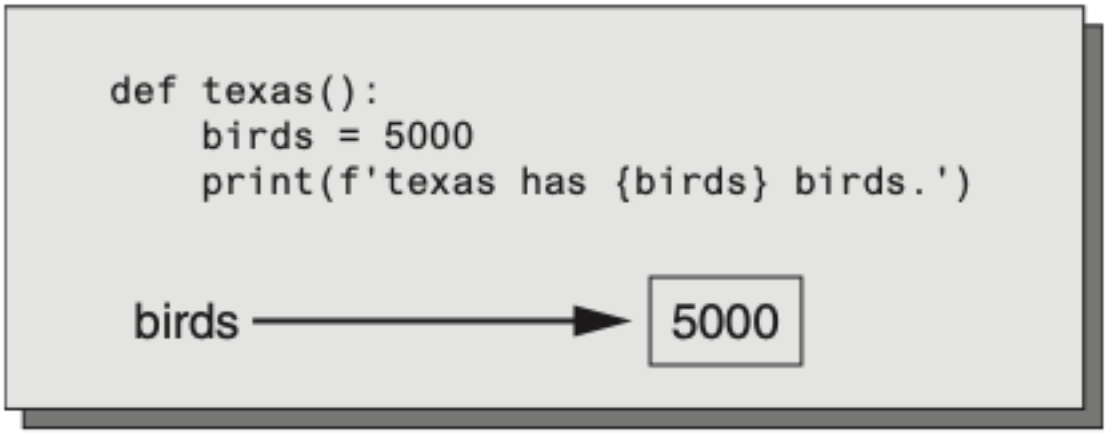
```
# This program demonstrates two functions that have local variables with the same name.
def main():
    # Call the texas function.
    texas()
    # Call the california function.
    california()

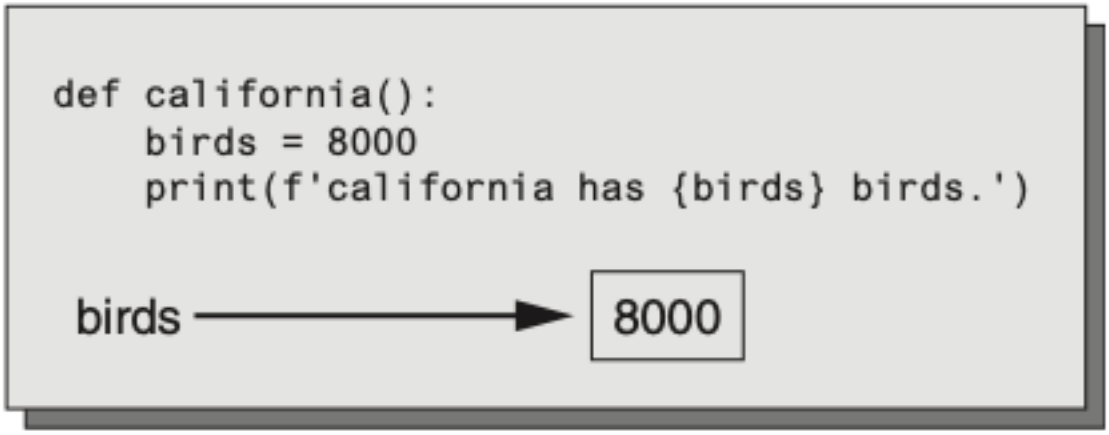
# Definition of the texas function. It creates a local variable named birds.
def texas():
    birds = 5000
    print('texas has', birds, 'birds.')
```



```
# Definition of the california function. It also creates a local variable named birds.
def california():
    birds = 8000
    print('california has', birds, 'birds.')

# Call the main function.
main()
```





222

223 Passing Arguments to Functions

224 An argument is any piece of data that is passed into a function when the function is called. A
225 parameter is a variable that receives an argument that is passed into a function.

226

- Sometimes it is useful not only to call a function, but also to send one or more pieces of data into the function.
- Pieces of data that are sent into a function are known as arguments.
- The function can use its arguments in calculations or other operations.
- If you want a function to receive arguments when it is called, you must equip the function with one or more parameter variables.
- A parameter variable, often simply called a parameter, is a special variable that is assigned the value of an argument when a function is called.

```
def show_double(number):
  result = number * 2
  print(result)
```

- This function's name is show_double.
- Its purpose is to accept a number as an argument and display the value of that number doubled.
- Look at the function header and notice the word number that appear inside the parentheses.
- This is the name of a parameter variable.
- This variable will be assigned the value of an argument when the function is called.

```
# This program demonstrates an argument being passed to a function.

# Define main function
def main():
    value = 5
    show_double(value)

# The show_double function accepts an argument and displays double its value.

# Define show_double
def show_double(number):
    result = number * 2
    print(result)

# Call the main function
main()
```

- When this program runs, the main function is called.
- Inside the main function, a local variable named value created, assigned the value 5.
- Then the following statement calls the show_double function:

```
show_double(value)
```

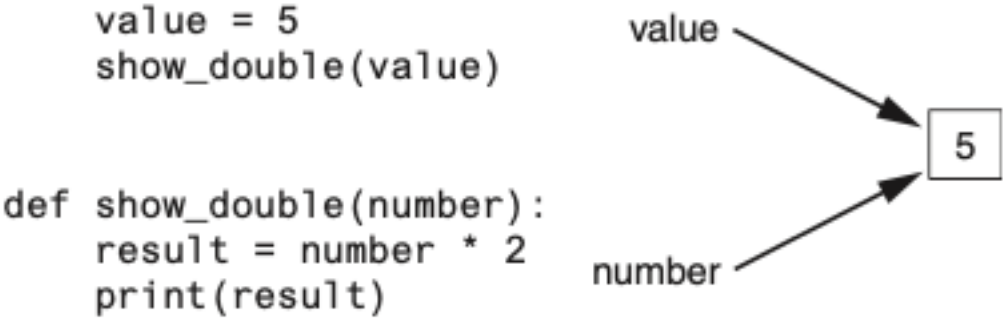
- Notice value appears inside the parentheses.
- This means that value is being passed as an argument to the show_double function

- When this statement executes, the show_double function will be called, and the number parameter will be assigned the same value as the value variable.

- Let's step through the show_double function.
- As we do, remember that the number parameter variable will be assigned the value that was passed to it as an argument.
- In this program, that number is 5.

```
def main():
    value = 5
    show_double(value)


def show_double(number):
    result = number * 2
    print(result)
```



- Line 11 assigns the value of the expression number * 2 to a local variable named result.
- Because number references the value 5, this statement assigns 10 to result.
- Line 12 displays the result variable.
- The following statement shows how the show_double function can be called with a numeric literal passed as an argument:


```
show_double(50)
```

- This statement executes the show_double function, assigning 50 to the number parameter. The function will print 100.

Parameter Variable Scope

- A variable is visible only to statements inside the variable's scope.
- A parameter variable's scope is the function in which the parameter is used.
- All of the statements inside the function can access the parameter variable, but no statement outside the function can access it.

Passing Multiple Arguments

- Often it's useful to write functions that can accept multiple arguments.
- Following code a function named show_sum, that accepts two arguments.
- The function adds the two arguments and displays their sum.

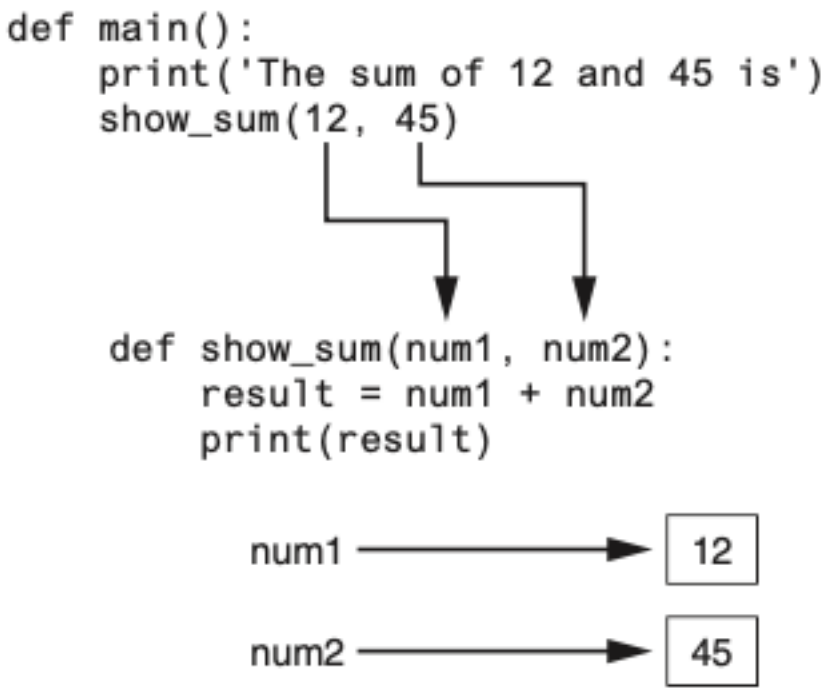
```
def main():
    print('The sum of 12 and 45 is ')
    show_sum(12, 45)

# show_sum
def show_sum(num1, num2):
    result = num1 + num2
    print(result)

# main()
main()
```

- Notice two parameter variable names, num1 and num2, appear inside the parentheses in the show_sum function header.
- This is often referred to as a parameter list. Also notice a comma separates the variable names.
- The statement in line 6 calls the show_sum function and passes two arguments: 12 and 45.
- These arguments are passed by position to the corresponding parameter variables in the function.

  In other words, the first argument is passed to the first parameter variable, and the second argument is passed to the second parameter variable. So, this statement causes 12 to be assigned to the num1 parameter and 45 to be assigned to the num2 parameter



- Suppose we were to reverse the order in which the arguments are listed in the function call, as shown here:

```
show_sum(45, 12)
```

- This would cause 45 to be passed to the num1 parameter, and 12 to be passed to the num2 parameter.
- This time, we are passing variables as arguments.

```
value1 = 2
value2 = 3
show_sum(value1, value2)
```

- When the show_sum function executes as a result of this code, the num1 parameter will be assigned the value 2, and the num2 parameter will be assigned the value 3.

Making Changes to Parameters

- When an argument is passed to a function in Python, the function parameter variable will reference the argument's value.
- However, any changes that are made to the parameter variable will not affect the argument.


```
def main():
    value = 99
    print('The value is', value)
    change_me(value)
    print('Back in main value is', value)

def change_me(arg):
    print('I am change the value.')
    arg = 0
    print('Now the valu is', arg)

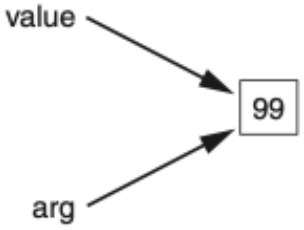
main()
```

```
The value is 99
I am change the value.
Now the valu is 0
Back in main value is 99
```

- The main function creates a local variable named value in line 5, assigned the value 99.
- The statement in line 6 displays 'The value is 99'. The value variable is then passed as an argument to the change_me function.
- This means that in the change_me function, the arg parameter will also reference the value 99.

```
def main():
    value = 99
    print(f'The value is {value}.')
    change_me(value)
    print(f'Back in main the value is {value}.')

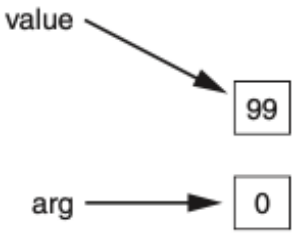

def change_me(arg):
    print('I am changing the value.')
    arg = 0
    print(f'Now the value is {arg}.')
```



- Inside the change_me function, in line 12, the arg parameter is assigned the value 0.
- This reassignment changes arg, but it does not affect the value variable in main.
- The two variables now reference different values in memory.
- The statement in line 13 displays 'Now the value is 0.' and the function ends.
- Control of the program then returns to the main function.
- The next statement to execute is in line 8.
- This statement displays 'Back in main the value is 99.'
- This proves that even though the parameter variable arg was changed in the change_me function, the argument (the value variable in main) was not modified.


```
def main():
    value = 99
    print(f'The value is {value}.')
    change_me(value)
    print(f'Back in main the value is {value}.')


def change_me(arg):
    print('I am changing the value.')
    arg = 0
    print(f'Now the value is {arg}.')
```



- The form of argument passing that is used in Python, where a function cannot change the value of an argument that was passed to it, is commonly called pass by value.
- This is a way that one function can communicate with another function.
- The communication channel works in only one direction, however.
- The calling function can communicate with the called function, but the called function cannot use the argument to communicate with the calling function.

Keyword Arguments

- Most programming languages match function arguments and parameters this way.
- In addition to this conventional form of argument passing, the Python language allows you to write an argument in the following format, to specify which parameter variable the argument should be passed to:

```
parameter_name=value
```

- In this format, parameter_name is the name of a parameter variable, and value is the value being passed to that parameter.
- An argument that is written in accordance with this syntax is known as a keyword argument.

- This program uses a function named show_interest that displays the amount of simple interest earned by a bank account for a number of periods.
- The function accepts the arguments principal (for the account principal), rate (for the interest rate per period), and periods (for the number of periods).
- When the function is called in line 7, the arguments are passed as keyword arguments.


```
def main():
    # Show the amount of simple interest, using 0.01 as
    # interest rate per period, 10 as the number of periods,
    # and $10,000 as the principal.
    show_interest(rate = 0.01, periods = 10, principal = 10000.0)

# The show_interest function displays the amount of simple interest for a given principal,
# interest rate, per period, and number of periods

def show_interest(principal, rate, periods):
    interest = principal * rate * periods
    print('The simple interest will be $',
    format(interest, ',.2f'),
    sep = "")

# Call the main function
main()
```

- Notice in line 7 the order of the keyword arguments does not match the order of the parameters in the function header in line 13.
- Because a keyword argument specifies which parameter the argument should be passed into, its position in the function call does not matter.

Mixing Keyword Arguments with Positional Arguments

- It is possible to mix positional arguments and keyword arguments in a function call, but the positional arguments must appear first, followed by the keyword arguments.
- Otherwise, an error will occur.
- Here is an example of how we might call the show_interest function of Program 5-10 using both positional and keyword arguments:

```
show_interest(10000.0, rate=0.01, periods=10)
```

- The first argument, 10000.0, is passed by its position to the principal parameter.
- The second and third arguments are passed as keyword arguments.
- The following function call will cause an error, however, because a non-keyword argument follows a keyword argument:

```
# This will cause an ERROR!
show_interest(1000.0, rate=0.01, 10)
```


Global Variables and Global Constants

A global variable is accessible to all the functions in a program file.

- Local Variable:
  1. Created by an assignment statement inside a function.
  2. Accessible only within the function that created it.
- Global Variable:
  1. Created by an assignment statement outside all functions in a program file.
  2. Accessible by any statement in the program file, including statements in any function.
- Best Practices:
  1. Restrict the use of global variables.
  2. Most programmers agree on minimizing or avoiding the use of global variables.
- Example (Program 5-13):
  1. Line 2: Creates a global variable named number.
  2. Line 5: Inside the main function, the global keyword is used to declare the number variable.
  3. Line 6: The value entered by the user is assigned to the global number variable.

```
# Create a global variable
my_value = 10

# The show_value function prints the value of the global variable
def show_value():
    print(my_value)

# Call the show_value function
show_value()
```

The reasons for use function

- Debugging Issues:
  - Global variables can be modified by any statement in the program.
  - Debugging is challenging because you need to track every statement that accesses the global variable to find where an incorrect value is being assigned.
  - This becomes increasingly difficult in large programs with thousands of lines of code.
- Dependency Problems:

  - Functions that use global variables depend on those variables.
  - Reusing such functions in different programs typically requires redesign to remove reliance on global variables.

- Understanding Complexity:

  - Global variables make a program harder to understand.
  - Any statement in the program can modify a global variable.
  - To understand a segment of the program involving a global variable, you must also understand all other segments that access it.

- Best Practice:

  - Create variables locally.
  - Pass variables as arguments to functions that need to access them.

Global Constants

- **Use of Global Constants**:

  1. It is permissible to use global constants in a program.
  2. A global constant is a global name referencing a value that cannot be changed.

- **Benefits of Global Constants**:

  1. The value of a global constant cannot be changed during the program's execution.
  2. This eliminates many potential hazards associated with global variables.

- **Python and Global Constants**:

  1. Python does not support true global constants.
  2. You can simulate global constants using global variables.

- **Simulation Method**:

  1. Do not declare the global variable with the `global` keyword inside a function.
  2. This prevents changing the variable's assignment inside that function.

- **Example**:

  1. The "In the Spotlight" section demonstrates how to use global variables in Python to simulate global constants.

Introduction to Value-Returning Functions: Generating Random Numbers

A value-returning function is a function that returns a value back to the part of the program that called it. Python, as well as most other programming languages, provides a library of prewritten functions that perform commonly needed tasks. These libraries typically contain a function that generates random numbers.

- **Void Functions**:
  1. A group of statements that exist within a program to perform a specific task.
  2. Called when the function needs to execute its task.
  3. Control returns to the statement immediately after the function call once the function is finished.
- **Value-Returning Functions**:
  1. Similarities to Void Functions:
     - A group of statements that perform a specific task.
     - Called when you want to execute the function.
  2. Differences from Void Functions:
     - Returns a value back to the part of the program that called it.
  3. The returned value can be:
     - Assigned to a variable.
     - Displayed on the screen.
     - Used in a mathematical expression (if it is a number).
     - Used like any other value in the program.

Standard Libary Functions and the import Statement

- **Standard Library Functions**:
  1. Python and most programming languages come with a standard library of pre-written functions.
  2. These functions, known as library functions, simplify a programmer's job by performing common tasks.
  3. Examples of Python library functions include `print`, `input`, and `range`.
- **Built-in Functions**:
  1. Some library functions are built into the Python interpreter.
  2. Examples of built-in functions include `print`, `input`, and `range`.
  3. Built-in functions can be called directly without additional steps.
- **Module-Based Functions**:
  1. Many standard library functions are stored in modules, which are files copied to your computer when Python is installed.
  2. Modules help organize functions with similar purposes, such as math operations or file handling.
  3. To use a module-based function, you need to write an import statement at the top of your program.
- **Import Statement**:
  1. An import statement tells the interpreter which module contains the functions you want to use.

2. Example: To use functions from the `math` module, write `import math` at the top of your program.
3. This statement loads the math module into memory and makes its functions available to the program.

- **Black Box Concept**:

1. Library functions can be thought of as "black boxes."
2. A "black box" accepts input, performs an unseen operation, and produces output.
3. The internal workings of library functions are not visible, simplifying their use.

Generating Random Numbers

- **Uses of Random Numbers**:

1. **Games**:
   - Used to simulate dice rolls or card draws in computer games.
2. **Simulation Programs**:
   - Used to randomly decide behaviors of living beings (people, animals, insects).
   - Formulas incorporating random numbers determine actions and events.
3. **Statistical Programs**:
   - Used to randomly select data for analysis.
4. **Computer Security**:
   - Used to encrypt sensitive data.

- **Random Module in Python**:

1. Python provides several library functions for working with random numbers.
2. These functions are stored in the `random` module in the standard library.
3. To use these functions, write the import statement at the top of your program:

```
import random
```

4. This statement loads the contents of the `random` module into memory, making its functions available to your program.

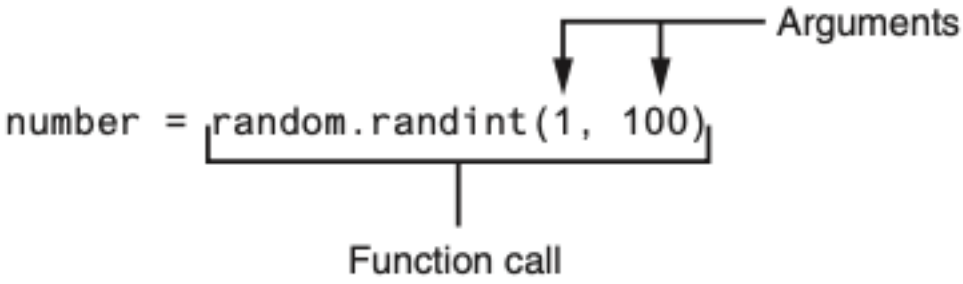
- **randint Function**:

1. The `randint` function generates random integers.

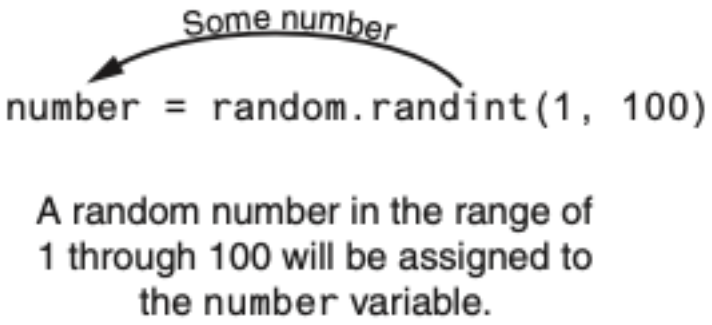
2. Because `randint` is in the `random` module, use dot notation to refer to it: `random.randint`.
3. Example call to `randint`:

```
number = random.randint(1, 100)
```

4. This statement calls the `randint` function with arguments `1` and `100`.
   - It generates a random integer in the range of 1 through 100 (inclusive).



- Notice the call to the randint function appears on the right side of an = operator.
- When the function is called, it will generate a random number in the range of 1 through 100 then return that number.
- The number that is returned will be assigned to the number variable



- **Program Example (Program 5-16)**:

1. **Description**:
   - Generates a random number in the range of 1 through 10.
   - Assigns the random number to the `number` variable.
2. **Example Statement**:

```
number = random.randint(1, 10)
```

   - Generates a random number between 1 and 10.
   - Assigns the value to the `number` variable (e.g., 7).

- **Direct Display of Random Numbers**:
  1. You can directly display a random number without assigning it to a variable.
  2. Example Statement:

```
print(random.randint(1, 10))
```

     - Calls `randint(1, 10)` to generate a random number between 1 and 10.
     - Sends the generated random number directly to the `print` function.
     - Displays a random number in the specified range.

- **How It Works**:
  1. The `randint` function is called.
  2. It generates a random number in the specified range (1 through 10).
  3. The generated number is returned and immediately sent to the `print` function.
  4. The random number is displayed on the screen.

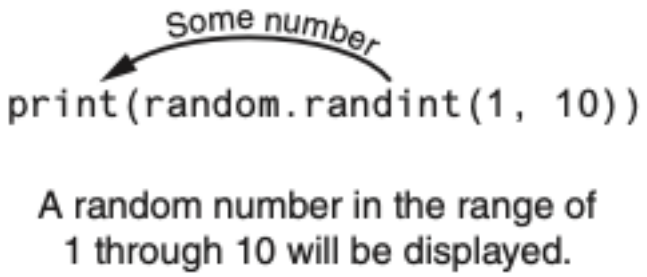
- **Figure 5-22 Illustration**:
  - Illustrates the process of generating and displaying a random number using the `randint` function and `print` function.

Example Program Code (Program 5-16)

```
import random

# Generate a random number between 1 and 10
number = random.randint(1, 10)

# Display the random number
print("The random number is:", number)
```



Calling Functions from an F-String

- **Function Call in F-Strings**:
  1. You can use a function call as a placeholder in an f-string.
  2. Example usage:

```
print(f'The number is {random.randint(1, 100)}.')
```

     - Generates a random number between 1 and 100.
     - Displays the message with the generated random number (e.g., "The number is 58").

- **Formatting Function Results with F-Strings**:
  1. F-strings are useful for formatting the result of a function call.
  2. Example for center-aligning a random number in a 10-character wide field:

```
print(f'{random.randint(0, 1000):^10d}')
```

     - Generates a random number between 0 and 1000.
     - Prints the number center-aligned in a field that is 10 characters wide.

Example Code

```
import random

# Example 1: Function call as a placeholder in an f-string
print(f'The number is {random.randint(1, 100)}.')

# Example 2: Formatting function result with an f-string
print(f'{random.randint(0, 1000):^10d}')
```

Experimenting with Random Numbers in Interactive Mode

- **Interactive Mode Example with `randint` Function**:
  1. **Interactive Session**: markdown 1 >>> import random # Enter to import the random module 2 >>> random.randint(1, 10) # Enter to generate a random number between 1 and 10 3 5 # Random number generated and displayed 4 >>> random.randint(1, 100) # Enter to generate a random number between 1 and 100 5 98 # Random number generated and displayed 6 >>> random.randint(100, 200) # Enter to generate a random number between 100 and 200 7 181 # Random number generated and displayed 8 >>>
  2. **Explanation**:

  - **Line 1**: The `import random` statement imports the `random` module.
    - You have to write the appropriate import statements in interactive mode.
  - **Line 2**: Calls the `randint` function, passing `1` and `10` as arguments.
    - The function returns a random number in the range of 1 through 10.
  - **Line 3**: The generated number (e.g., 5) is displayed.
  - **Line 4**: Calls the `randint` function, passing `1` and `100` as arguments.
    - The function returns a random number in the range of 1 through 100.
  - **Line 5**: The generated number (e.g., 98) is displayed.
  - **Line 6**: Calls the `randint` function, passing `100` and `200` as arguments.
    - The function returns a random number in the range of 100 through 200.
  - **Line 7**: The generated number (e.g., 181) is displayed.

- **Usage of `randint` Function**:

  1. **Assignment to Variable**:
     - You can assign the function's return value to a variable.
     - Example:

```
number = random.randint(1, 10)
```

  2. **Direct Printing**:
     - You can directly print the function's return value.
     - Example:

```
print(random.randint(1, 10))
```

  3. **In Math Expressions**:
     - Example:

```
x = random.randint(1, 10) * 2
```

       - Generates a random number between 1 and 10.
       - Multiplies the random number by 2.
       - Assigns the result (an even integer between 2 and 20) to variable `x`.

- **Conditional Testing**:

  - You can test the return value of the function with an `if` statement.
  - This allows for conditional operations based on the generated random number.

Example Code

```
import random

# Example 1: Interactive mode snippets
number1 = random.randint(1, 10)  # Generates a random number between 1 and 10
```



```
print(number1)  # Displays the generated number

number2 = random.randint(1, 100)  # Generates a random number between 1 and 100
print(number2)  # Displays the generated number

number3 = random.randint(100, 200)  # Generates a random number between 100 and 200
print(number3)  # Displays the generated number

# Example 2: Using randint function in a math expression
x = random.randint(1, 10) * 2
print(f'Random even number between 2 and 20: {x}')

# Example 3: Conditional testing with randint
if random.randint(1, 2) == 1:
    print("Heads")
else:
    print("Tails")
```

The randrnage, random, and uniform function

- **Additional Functions in the `random` Module**:
    1. **Importing the `random` Module**:

       ```
       import random
       ```

- **`randrange` Function**:
    1. Similar to the `range` function but returns a randomly selected value.
    2. Usage Examples:
        - **Single Argument**:

          ```
          number = random.randrange(10)
          ```

            * Returns a random number from 0 up to, but not including, 10.
        - **Starting and Ending Values**:

          ```
          number = random.randrange(5, 10)
          ```

            * Returns a random number from 5 up to, but not including, 10.
        - **Starting Value, Ending Limit, and Step Value**:

          ```
          number = random.randrange(0, 101, 10)
          ```

            * Returns a random number from the sequence: `[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]`

- **`random` Function**:

1. Returns a random floating-point number.
2. No arguments needed.
3. Usage Example:

```
number = random.random()
```

   - Returns a random floating-point number in the range of 0.0 up to 1.0 (not including 1.0).

- **`uniform` Function**:

1. Returns a random floating-point number within a specified range.
2. Usage Example:

```
number = random.uniform(1.0, 10.0)
```

   - Returns a random floating-point number in the range of 1.0 through 10.0.

Example Code

```
import random

# Example usage of `randrange` function
number1 = random.randrange(10)
print(f'Random number between 0 and 9: {number1}')

number2 = random.randrange(5, 10)
print(f'Random number between 5 and 9: {number2}')

number3 = random.randrange(0, 101, 10)
print(f'Random number from sequence [0, 10, ..., 100]: {number3}')

# Example usage of `random` function
number4 = random.random()
print(f'Random floating-point number between 0.0 and 1.0: {number4}')

# Example usage of `uniform` function
number5 = random.uniform(1.0, 10.0)
print(f'Random floating-point number between 1.0 and 10.0: {number5}')
```

Random Number Seeds

- **Pseudorandom Numbers**:

1. The numbers generated by the functions in the `random` module are not truly random.

2. They are called pseudorandom numbers and are calculated using a formula.

- **Seed Value**:

1. The formula that generates random numbers is initialized with a seed value.
2. The seed value is used in the calculation that returns the next random number in the series.
3. When the `random` module is imported, it uses the system time (current date and time down to a hundredth of a second) as the seed value.
4. This ensures different sequences of random numbers each time the module is imported.

- **Custom Seed Value**:

1. You can specify a custom seed value using the `random.seed` function.
2. Example:

```
random.seed(10)
```

   – Sets 10 as the seed value.

- **Reproducible Pseudorandom Numbers**:

1. Using the same seed value will always generate the same sequence of pseudorandom numbers.
2. Example interactive session:

```
>>> import random  # Import the random module
>>> random.seed(10)  # Set seed value to 10
>>> random.randint(1, 100)  # Generate random number between 1 and 100
58  # Output
>>> random.randint(1, 100)
43  # Output
>>> random.randint(1, 100)
58  # Output
>>> random.randint(1, 100)
21  # Output
```

Example Code

```
import random

# Setting a custom seed value
random.seed(10)

# Generating reproducible pseudorandom numbers
number1 = random.randint(1, 100)
print(f'First random number (seed 10): {number1}')
```



```
number2 = random.randint(1, 100)
print(f'Second random number (seed 10): {number2}')

number3 = random.randint(1, 100)
print(f'Third random number (seed 10): {number3}')

number4 = random.randint(1, 100)
print(f'Fourth random number (seed 10): {number4}')
```

Writing Your Own Value-Returning Functions

A value-returning function has a return statement that returns a value back to the part of the program that called it.

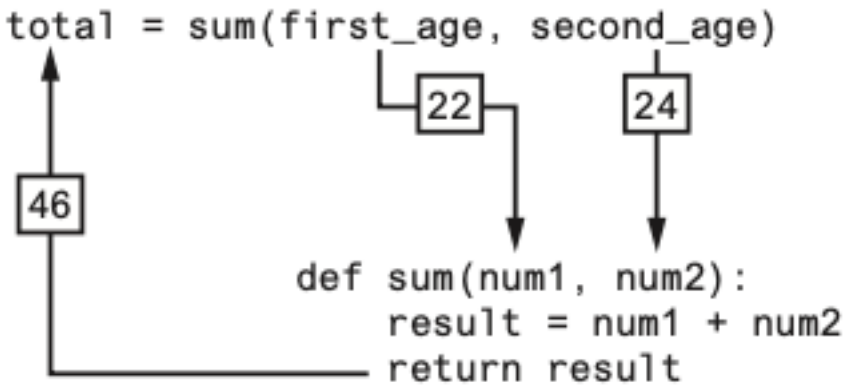
- **Value-Returning Function**:
    1. Written similarly to a void function but must include a `return` statement.
- **General Format**: “‘python def function_name(): statement statement # Other statements return expression

```
![](img/21.png)

### Purpose of the `sum` Function
- **Accepts Two Integer Values**: Takes two integer arguments and returns their sum.
- **How It Works**:
  1. The first statement assigns the value of `num1 + num2` to the `result` variable.
  2. The `return` statement sends the value of `result` back to the calling part of the program,

### Example Implementation
```python
def sum(num1, num2):
    result = num1 + num2
    return result
```


```




### Making the Most of the return Statement

- Two Steps Inside the Function:
  - The expression num1 + num2 is assigned to the result variable.
  - The value of the result variable is returned.

#### Simplified `sum` Function

- **Original Version (Program 5-21)**:

```
def sum(num1, num2):
    result = num1 + num2
    return result
```

How to Use Value-Returning Functions

Benefits of Value-Returning Functions

- **Simplify Code**: Makes code easier to read and understand.
- **Reduce Duplication**: Prevents repetitive code.
- **Enhance Testing**: Easier to test individual components.
- **Increase Development Speed**: Streamlines the coding process.
- **Facilitate Teamwork**: Simplifies collaboration by modularizing code.

Example: User Input Function

- **Purpose**: Prompt the user for input and return the value.
- **Function Definition**:

```
def get_regular_price():
    price = float(input("Enter the item's regular price: "))
    return price
```

- Value-Returning Functions: Useful for handling user input and simplifying complex calculations.
- Modular Code: Breaking down tasks into functions enhances readability and maintainability.
- Practical Application: Used here to calculate the sale price of an item with a discount.

Using IPO Charts

IPO Chart for Designing Functions

An IPO (Input, Processing, Output) chart is a useful tool for designing and documenting functions.

- **Input**: Describes the data passed to the function as arguments.
- **Processing**: Describes the operation or process the function performs.
- **Output**: Describes the data that the function returns.

IPO Chart Examples

IPO Chart for `get_regular_price` Function

Input	Processing	Output
None	Prompts user for item's regular price and converts it to float	Returns the entered price as a float

IPO Chart for `discount` Function

Input	Processing	Output
price (float)	Computes discount as `price` multiplied by `DISCOUNT_PERCENTAGE`	Returns the discount amount as a float

635 **Example Functions (Program 5-22)**

636 `get_regular_price` **Function**

```
def get_regular_price():
    price = float(input("Enter the item's regular price: "))
    return price
```

The `get_regular_price` Function		
Input	Processing	Output
None	Prompts the user to enter an item's regular price	The item's regular price

The `discount` Function		
Input	Processing	Output
An item's regular price	Calculates an item's discount by multiplying the regular price by the global constant `DISCOUNT_PERCENTAGE`	The item's discount

637

638 • Notice the IPO charts provide only brief descriptions of a function's input, processing, and output,
639 but do not show the specific steps taken in a function.

- In many cases, however, IPO charts include sufficient information so they can be used instead of a flowchart.
- The decision of whether to use an IPO chart, a flowchart, or both is often left to the programmer's personal preference.

Returning Strings

Functions Returning Strings

- **Functions can return** different data types, including strings.
- **Examples** will demonstrate functions returning user-entered strings and f-strings with formatted values.

Function Returning a User-Entered String

- **Function Definition**:

```
def get_name():
    # Get the user's name.
    name = input('Enter your name: ')
    # Return the name.
    return name
```

- Functions Returning Strings:
  - Can prompt the user for input and return the input as a string.
  - Can return f-strings with formatted values.
- Practical Application:
  - get_name(): Returns a user-entered name.
  - dollar_format(value): Returns a value formatted as a dollar amount.
- Example Program:
  - Demonstrates how to use both functions in a main function and display the results.

Returning Boolean Values

Boolean Functions in Python

- **Boolean Functions**:
  - Return either `True` or `False`.
  - Used to test conditions and simplify complex decision and repetition structures.

Example: Determining Even or Odd Numbers

- **Previous Approach**:

```
number = int(input('Enter a number: '))
if (number % 2) == 0:
    print('The number is even.')
else:
    print('The number is odd.')
```

- Boolean Functions:
  - Simplify the logic by returning True or False based on conditions.
  - Can be reused in multiple parts of the program to test specific conditions.
- Example:
  - is_even function checks if a number is even.
  - The main function gets user input, calls is_even, and prints the result.
- Practical Benefits
  - Readability: Code is easier to read and understand.
  - Reusability: Functions can be reused wherever the same condition needs to be tested.
  - Maintainability: Easier to maintain and update the code when logic is encapsulated in functions.

Initial Validation Algorithm

```
# Get the model number.
model = int(input('Enter the model number: '))

# Validate the model number.
while model != 100 and model != 200 and model != 300:
    print('The valid model numbers are 100, 200, and 300.')
    model = int(input('Enter a valid model number: '))
```

- Boolean Functions: Useful for simplifying complex validation logic.
- Improved Readability: The validation loop is easier to understand when using a Boolean function.
- Reusable Logic: The Boolean function can be reused in different parts of the program for the same validation purpose.

Returning Multiple Values

Returning Multiple Values from a Function

Overview

- **Value-Returning Functions**: So far, examples have shown functions returning a single value.
- **Multiple Returns**: In Python, functions can return multiple values.

General Format for Returning Multiple Values

```
return expression1, expression2, etc.
```

- Multiple Value Returns: Python allows functions to return multiple values.
- Unpacking: When calling functions that return multiple values, you can unpack the results into multiple variables.
- Error Prevention: Ensure the number of variables matches the number of returned values to avoid errors.

Returning None from a Function

Overview

- `None`: A special built-in value in Python used to represent no value.
- **Use Case**: Returning `None` from a function can indicate that an error has occurred.

Example: Handling Division by Zero

- Initial Function (Without Error Handling)

```
def divide(num1, num2):
    return num1 / num2
```

- Returning None:
  - Useful for indicating errors, such as division by zero.
- Using the is Operator:
  - Preferable over == when comparing a variable to None.
- Example Program:
  - Demonstrates handling division by zero gracefully by returning None and checking the result.

- Best Practices:
  - Always use is and is not when comparing variables to None.

The math Module

The Python standard library's math module contains numerous functions that can be used in mathematical calculations.

Using the `math` Module in Python

Overview

- The `math` module in the Python standard library contains several functions for performing mathematical operations.
- Functions in this module typically accept one or more values as arguments, perform a mathematical operation, and return the result.
- Most functions return a float value, except for `ceil` and `floor`, which return int values.

Common Functions in the `math` Module

- All functions listed return a float value unless otherwise specified.
- They perform various mathematical operations.

Example: Utilizing the `sqrt` Function

- **Function**: `sqrt`
- **Purpose**: Returns the square root of the given argument.

Example Usage

```
import math

result = math.sqrt(16)
print(result) # Output: 4.0
```

- math Module: Contains various mathematical functions.
- Common Usage: Functions typically accept arguments, perform operations, and return float values (with exceptions for ceil and floor).

- Example Function (sqrt): Demonstrates how to compute the square root of a number using the math.sqrt function.
- Best Practices: Always import the math module when using its functions in your program.

math Module Function	Description
acos(x)	Returns the arc cosine of x, in radians.
asin(x)	Returns the arc sine of x, in radians.
atan(x)	Returns the arc tangent of x, in radians.
ceil(x)	Returns the smallest integer that is greater than or equal to x.
cos(x)	Returns the cosine of x in radians.
degrees(x)	Assuming x is an angle in radians, the function returns the angle converted to degrees.
exp(x)	Returns e^x
floor(x)	Returns the largest integer that is less than or equal to x.
hypot(x, y)	Returns the length of a hypotenuse that extends from (0, 0) to (x, y).
log(x)	Returns the natural logarithm of x.
log10(x)	Returns the base-10 logarithm of x.
radians(x)	Assuming x is an angle in degrees, the function returns the angle converted to radians.
sin(x)	Returns the sine of x in radians.
sqrt(x)	Returns the square root of x.
tan(x)	Returns the tangent of x in radians

The math.pi and math.e Values

- The math module also defines two variables, pi and e, which are assigned mathematical values for pi and e.
- You can use these variables in equations that require their values.
- For example, the following statement, which calculates the area of a circle, uses pi. (Notice we use dot notation to refer to the variable.)
  - area = math.pi * radius**2

Storing Functions in Modules

A module is a file that contains Python code. Large programs are easier to debug and maintain when they are divided into modules.

Organizing Code with Modules

Overview

- **Modules**: Files that contain Python code, typically organized to perform related tasks.
- **Modularization**: Breaking a program into modules to improve readability, testing, and maintenance.

Benefits of Modularization

1. **Organized Code**: Functions related to specific tasks are grouped together.
2. **Reusability**: Code in modules can be reused across multiple programs.
3. **Maintainability**: Easier to manage and update code.

Example Scenario

- **Task**: Write a program to calculate:
  - The area of a circle
  - The circumference of a circle
  - The area of a rectangle
  - The perimeter of a rectangle

Creating and Using Modules

1. **Circle Module** (`circle.py`):
   - **Content**:

```python
import math

def area(radius):
    return math.pi * radius ** 2

def circumference(radius):
    return 2 * math.pi * radius
```

2. **Rectangle Module** (`rectangle.py`):
   - **Content**:

```python
def area(width, height):
    return width * height

def perimeter(width, height):
    return 2 * (width + height)
```


Using the Modules in a Main Program

1. **Main Program** (`main.py`):

   - **Content**:

```python
import circle
import rectangle

def main():
    # Demonstrate circle calculations
    radius = float(input("Enter the radius of the circle: "))
    print(f"Area of the circle: {circle.area(radius)}")
    print(f"Circumference of the circle: {circle.circumference(radius)}")

    # Demonstrate rectangle calculations
    width = float(input("Enter the width of the rectangle: "))
    height = float(input("Enter the height of the rectangle: "))
    print(f"Area of the rectangle: {rectangle.area(width, height)}")
    print(f"Perimeter of the rectangle: {rectangle.perimeter(width, height)}")

if __name__ == "__main__":
    main()
```

Important Notes on Module Names

- **File Extension**: Module file names should end in `.py`.
- **Naming Constraints**: Module names cannot be the same as Python keywords.

Importing Modules

- **Import Statement**: To use a module, import it using the `import` statement:

```python
import circle
```

- Modules: Enable organized, maintainable, and reusable code.
- Modularization: Group related functions into modules to simplify complex programs.
- Usage: Import modules to utilize the functions within them in your main program.
- Best Practices: Ensure module names end with .py and do not clash with Python keywords.

- Once a module is imported you can call its functions. Assuming radius is a variable that is assigned the radius of a circle, here is an example of how we would call the area and circumference functions:


```
my_area = circle.area(radius)
my_circum = circle.circumference(radius)
```

Conditionally Executing the main Function in a Module

Understanding `__name__` and Module Imports

Overview

- When a module is imported, Python executes its statements as if it were a standalone program.
- Modules typically define functions and other elements intended to be used by other programs.

Importing Modules

- **Example with `circle.py`**:
  - Imports the `math` module.
  - Defines functions: `area` and `circumference`.
- **Example with `rectangle.py`**:
  - Defines functions: `area` and `perimeter`.

Executing Modules as Standalone Programs

- **Typical Use Case**: Modules are meant to be imported, not run directly.
- **Possible Scenario**: Sometimes you want a module to be runnable as a standalone program or be importable without executing its main function.

Special Variable `__name__`

- Python creates a special variable `__name__` during the execution of a source file.
- **Values of `__name__`**:
  - If the file is being run directly, `__name__` is set to `'__main__'`.
  - If the file is imported as a module, `__name__` is set to the module's name.

Conditional Main Function Execution

- **Purpose**: To determine whether the file is being executed directly or imported as a module.
- **Technique**: Use the value of `__name__` to control execution.

```
if __name__ == "__main__":
    main()
```

- This ensures:
  - The main function runs only if the file is executed directly.
  - The main function does not run if the file is imported as a module.

- '**name**' Variable: Allows you to control whether the main function runs based on how the file is executed.
- Best Practice: Use if '**name**' == "**main**": to ensure a file behaves correctly whether executed directly or imported.
- Modular Design: Facilitates reusable, organized, and manageable code.
- This technique is recommended for use in any Python source file containing a main function, ensuring appropriate behavior whether the file is run as a standalone program or imported as a module.

Turtle Graphics: Modularizing Code with Functions

Commonly needed turtle graphics operations can be stored in functions and then called whenever needed.

Using Turtle Graphics to Draw Shapes with Functions

Overview

- Drawing shapes with Turtle involves several steps.
- Writing repetitive code can be simplified using functions.
- Functions allow us to draw shapes by specifying parameters like coordinates, dimensions, and colors.

Example 1: Drawing a Square

Function Definition


```
import turtle

def square(x, y, width, color):
    turtle.penup()
    turtle.goto(x, y)
    turtle.pendown()
    turtle.fillcolor(color)
    turtle.begin_fill()
    for count in range(4):
        turtle.forward(width)
        turtle.left(90)
    turtle.end_fill()
```

```
def main():
    # Draw three squares by calling the square function
    square(100, 0, 50, 'red')
    square(-150, -100, 200, 'blue')
    square(-200, 150, 75, 'green')
    turtle.done()

if __name__ == "__main__":
    main()
```

- Square Function:
  - Parameters: x, y (coordinates of the lower-left corner), width (size of the sides), and color.
  - Draws a square at the specified location with the specified width and color by using Turtle commands.
- Circle Function:
  - Parameters: x, y (coordinates of the center), radius, and color.
  - Draws a circle at the specified location with the specified radius and color.
- Line Function:
  - Parameters: startX, startY (coordinates of the starting point), endX, endY (coordinates of the ending point), and color.
  - Draws a line from the starting point to the ending point with the specified color.

- Functions: Modularize code for repetitive drawing tasks, making it easier to call specific shapes with given parameters.
- Squares: square(x, y, width, color) draws a square.
- Circles: circle(x, y, radius, color) draws a circle.
- Lines: line(startX, startY, endX, endY, color) draws a line.
- Best Practices: Utilizing functions for drawing with Turtle enhances code reusability and readability.

838 ## Storing Your Graphics Functions in a Module

839 ## Storing Turtle Graphics Functions in a Module

840 ### Overview

841 - Storing frequently used Turtle graphics functions in a module allows for easy reuse.
842 - You can import the module into any program that needs to use the functions.

843 ### Example: Creating the `my_graphics.py` Module

844 `my_graphics.py`

```
import turtle

def square(x, y, width, color):
    turtle.penup()
    turtle.goto(x, y)
    turtle.pendown()
    turtle.fillcolor(color)
    turtle.begin_fill()
    for count in range(4):
        turtle.forward(width)
        turtle.left(90)
    turtle.end_fill()

def circle(x, y, radius, color):
    turtle.penup()
    turtle.goto(x, y - radius)
    turtle.pendown()
    turtle.fillcolor(color)
    turtle.begin_fill()
    turtle.circle(radius)
    turtle.end_fill()

def line(startX, startY, endX, endY, color):
    turtle.penup()
    turtle.goto(startX, startY)
    turtle.pendown()
    turtle.pencolor(color)
    turtle.goto(endX, endY)
```



```
import turtle
import my_graphics

def main():
    # Draw shapes using functions from my_graphics module
    my_graphics.square(100, 0, 50, 'red')
    my_graphics.square(-150, -100, 200, 'blue')
    my_graphics.square(-200, 150, 75, 'green')

    my_graphics.circle(0, 0, 100, 'red')
    my_graphics.circle(-150, -75, 50, 'blue')
    my_graphics.circle(-200, 150, 75, 'green')

    my_graphics.line(0, 100, -100, -100, 'red')
    my_graphics.line(0, 100, 100, 100, 'blue')
    my_graphics.line(-100, -100, 100, 100, 'green')

    turtle.done()

if __name__ == "__main__":
    main()
```

Explanation

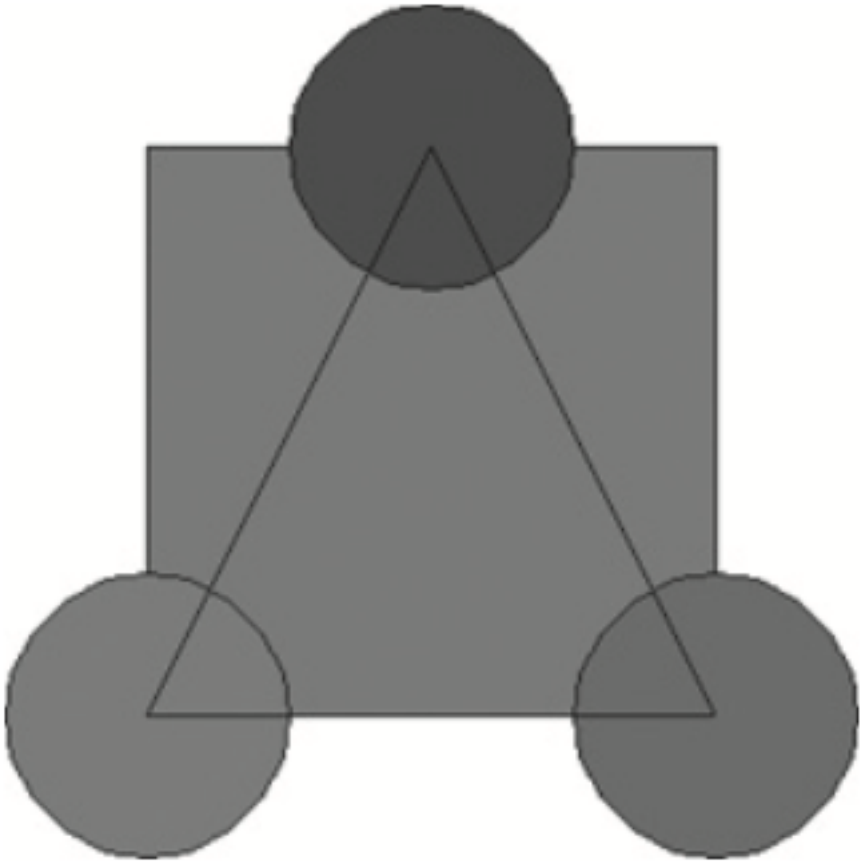
- Creating the Module (my_graphics.py):
  - The module defines three functions: square, circle, and line.
  - Each function contains the necessary Turtle commands to draw the respective shape.
- Using the Module:
  - Import the Module: The statement import my_graphics imports the module.
  - Call the Functions: Use my_graphics.function_name to call functions from the module.

Summary

- Modularization: Storing Turtle graphics functions in a module enhances reusability and organization.
- Importing Modules: You can import the module into any program that needs to draw shapes with Turtle graphics.
- Using Functions: Call the functions defined in the module to draw shapes as needed, simplifying your main program.

Benefits

- Reusability: Easily reuse functions across multiple programs.
- Maintainability: Manage and update your drawing functions in one place.
- Readability: Simplify your main program code by modularizing repetitive tasks.



House Keeping

Announcement

Assignment

Assignment1: Assignment Title

- **What to do**
  - A
  - B
- **Format and Requirement**
  - PDF format, < ?? pages
  - file name should be include your student id and name
    - stuID_name_title.pdf (e.g. 1111111_ChungilChae_SelfIntroduction.pdf)

875 – APA, Double-Space, No title page, 12 points, Reference section if necessary
876 – Chinese name in English, English name, Student ID, Class name and section (e.g. GE2021,
877 W09; MGS3001, W01)

878 • due date

879 – by Date(DAY) 11:59PM
880 – NO LATE SUBMISSION ALLOWED!!!!

881 **Reference**

